

sunSafePoint

1 Executive Summary

This module computes the attitude guidance output for a sun-pointing mode, suitable for safe mode or power generation. Given the sun direction vector (not necessarily normalized) and the body angular velocity, the module produces attitude tracking errors as Modified Rodrigues Parameters (MRPs) and body rate tracking errors. The sun direction measurement is crossed with a commanded body-fixed axis to obtain a principal rotation vector, and the dot product yields the principal rotation angle. If the sun vector is not available (the zero vector), the attitude error is zeroed and a configurable body-fixed search rate is used instead. An optional constant spin rate about the sun heading axis can be specified.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
sunDirectionInMsg	NavAttMsgF32Payload	input message containing the sun direction vector $\mathbf{s}$ in body frame coordinates
imuInMsg	NavAttMsgF32Payload	input message containing the inertial body angular velocity $\omega_{B/N}$
attGuidanceOutMsg	AttGuidMsgF32Payload	output message of attitude tracking errors and reference frame states

3 Module Parameters

The following table lists all the module parameters that can be set. Required parameters must be configured before the module is used.

Table 2: Module Parameters

Parameter Name	Type	Units	Default	Description	Bounds
sHatBdyCmd (required)	Eigen::Vector3f [-]		zero	Commanded body-fixed unit direction vector $\hat{\mathbf{s}}_c$ to align with the sun	Norm must be within $1\mathbf{e-3}$ of 1.0 (checked in setter; vector is renormalized on storage)
smallAngle (required)	float	[rad]	0	Angle threshold ϵ for detecting near-collinear vectors	Must be positive (checked in setter)
sunAxisSpinRate	float	[rad/s]	0	Desired constant spin rate $\dot{\theta}$ about the sun heading vector	None

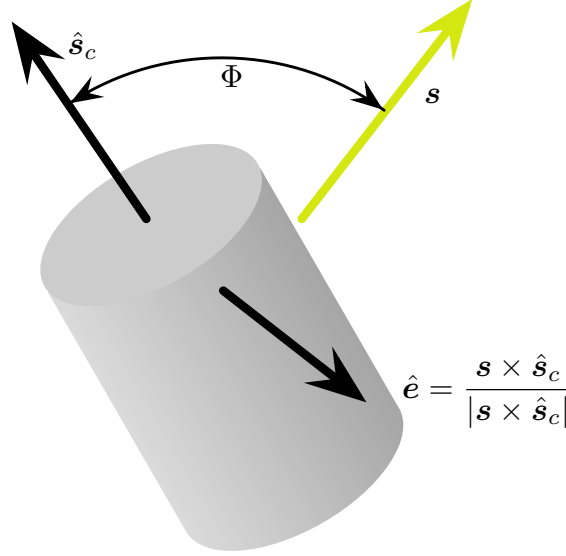


Figure 1: Body Vector Illustrations

Parameter Name	Type	Units	Default	Description	Bounds
omega_RN_B	Eigen::Vector3f	[rad/s]	zero	Body-fixed reference rate $\omega_{R/N}$ used when no sun direction is available	None

4 Module Assumptions and Limitations

- The input sun direction vector $\mathbf{s}$ can have any length. Only the exact zero vector is treated as sun-not-available; any non-zero magnitude is accepted (normalization uses `stableNormalized` for numerical robustness with small inputs).
- The commanded body-relative unit direction vector $\hat{\mathbf{s}}_c$ is assumed to be fixed relative to the body.
- The sun-pointing condition is under-determined (2 DOF): the rotation about $\mathbf{s}$ is arbitrary. This is acceptable for power generation, which does not depend on orientation about the sun heading.

5 Initialization

The module is configured by:

```

module = sunSafePointF32.SunSafePoint()
module.modelTag = "sunSafePoint"
module.sHatBdyCmd = [0.0, 0.0, 1.0]
module.smallAngle = 0.001
module.sunAxisSpinRate = 0.0
module.omega_RN_B = [0.0, 0.0, 0.0]

```

6 Detailed Module Description

6.1 Algorithm Flow

At every update cycle, the `sunSafePoint` module performs the following steps:

1. **Check sun visibility:** Normalize $\mathbf{s}$ using `stableNormalized`. If the result is the zero vector (i.e. the input was exactly zero), set $\sigma_{B/R} = \mathbf{0}$, use the configured `omega.RN_B`, and skip to step 5.
2. **Compute the principal rotation angle Φ** between $\mathbf{s}$ and $\hat{\mathbf{s}}_c$.
3. **Determine the eigen axis $\hat{\mathbf{e}}$:**
 - If $\Phi < \epsilon$: set $\sigma_{B/R} = \mathbf{0}$ (already aligned).
 - If $\pi - \Phi < \epsilon$: use a fallback axis $\hat{\mathbf{e}}_{180}$ orthogonal to $\hat{\mathbf{s}}_c$, computed via Eigen's `unitOrthogonal()`.
 - Otherwise: $\hat{\mathbf{e}} = (\mathbf{s} \times \hat{\mathbf{s}}_c) / |\mathbf{s} \times \hat{\mathbf{s}}_c|$.
4. **Compute attitude error MRP** $\sigma_{B/R} = \tan(\Phi/4) \hat{\mathbf{e}}$ and the reference rate $\omega_{R/N} = \dot{\theta} \hat{\mathbf{s}}$.
5. **Compute rate tracking error** $\omega_{B/R} = \omega_{B/N} - \omega_{R/N}$.

6.2 Equations

6.2.1 Good Sun Direction Vector Case

In the following mathematical developments all vectors are assumed to be taken with respect to a body-fixed frame $\mathcal{B}$. The attitude of the body $\mathcal{B}$ relative to the reference frame $\mathcal{R}$ is written as a principal rotation from $\mathcal{R}$ to $\mathcal{B}$. Thus, the associated principal rotation vector $\hat{\mathbf{e}}$ is

$$\hat{\mathbf{e}} = \frac{\mathbf{s} \times \hat{\mathbf{s}}_c}{|\mathbf{s} \times \hat{\mathbf{s}}_c|}$$

Note that the sun direction vector $\mathbf{s}$ does not have to be a normalized input vector.

The principal rotation angle between the two vectors is given through

$$\Phi = \arccos\left(\frac{\mathbf{s} \cdot \hat{\mathbf{s}}_c}{|\mathbf{s}|}\right)$$

Next, this rotation from $\mathcal{R}$ to $\mathcal{B}$ is written as a set of MRPs through

$$\sigma_{B/R} = \tan\left(\frac{\Phi}{4}\right) \hat{\mathbf{e}}$$

The set $\sigma_{B/R}$ is the attitude error of the output attitude guidance message.

The module allows for a nominal spin rate about the sun heading axis by specifying the module parameter `sunAxisSpinRate` called $\dot{\theta}$ in this description. The nominal spin rate is thus given by

$${}^{\mathcal{B}}\omega_{R/N} = {}^{\mathcal{B}}\hat{\mathbf{s}}\dot{\theta}$$

Note that this constant nominal spin is only for the case where the sun is visible and the sun-heading vector measurement is available.

If the spacecraft is to be brought to rest $\omega_{R/N} = \mathbf{0}$, then $\dot{\theta}$ should be set to zero. The tracking error angular velocity vector is computed using

$$\omega_{B/R} = \omega_{B/N} - \omega_{R/N}$$

Finally, the attitude guidance message must specify the inertial reference frame acceleration vector. This is set to zero as the roll about the sun heading is assumed to have a constant magnitude and inertial heading.

$$\dot{\omega}_{R/N} = \mathbf{0}$$

6.2.2 No Sun Direction Vector Case

If $\mathbf{s}$ is the zero vector, then it is assumed that no good sun heading vector is available and the attitude tracking error $\sigma_{B/R}$ is set to zero. Any non-zero magnitude is treated as a valid sun direction; `stableNormalized` is used to handle small inputs robustly.

Further, if the sun is not visible, the module allows for a non-zero body rate to be prescribed. This allows the spacecraft to engage in a constant rate tumble specified through the module configuration vector `omega_RN_B`. In this case the tracking error rate is evaluated through

$$\omega_{B/R} = \omega_{B/N} - \omega_{R/N}$$

and the output message reference rate is set equal to the prescribed $\omega_{R/N}$ while the reference acceleration vector $\dot{\omega}_{R/N}$ is set to zero.

6.2.3 Collinear Commanded and Sun Heading Vectors

First consider the case where $\mathbf{s} \approx \hat{\mathbf{s}}_c$. In this case the cross product is not well defined. Let ϵ be a pre-determined small angle. Then, if $\Phi < \epsilon$ the attitude tracking error is set to

$$\sigma_{B/R} = \mathbf{0}$$

However, if $\mathbf{s} \approx -\hat{\mathbf{s}}_c$, then an eigen-axis $\hat{\mathbf{e}}$ that is orthogonal to $\hat{\mathbf{s}}_c$ must be determined. The choice of axis is arbitrary (any vector perpendicular to $\hat{\mathbf{s}}_c$ produces a valid 180° rotation), so the module delegates to Eigen's `unitOrthogonal()`, which returns a unit vector perpendicular to $\hat{\mathbf{s}}_c$ using a numerically stable axis choice:

$$\hat{\mathbf{e}}_{180} = \mathbf{sHatBdyCmd.unitOrthogonal()}$$

This fallback axis is computed inline in `update()` only on iterations where $\pi - \Phi < \epsilon$.

In this scenario the angular velocity tracking error is evaluated using the same method as outlined in the Good Sun Direction Vector Case section.

7 Test Description

The module is verified through regression tests against an independently coded reference implementation, property-based tests covering MRP norm bounds, zero-error conditions, and the rate identity $\omega_{B/R} = \omega_{B/N} - \omega_{R/N}$, as well as edge-case tests for collinear vectors, 180-degree rotations, the zero-vector sun-not-visible case, and the $\hat{\mathbf{e}}_{180}$ fallback axis.